SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

StructRender

The Complete Project Guide

Everything we built, why, how it works, what we found, and how to explain it

Engine-Grounded Real-Time Neural Rendering of a Game in Learned Painterly Styles

Kabilash S – E0323036
Pavitra P – E0323038

Guide: Dr. V. Vanitha, Associate Professor, Department of CSE (AIML)
ADA23DLU06 – Recent Trends in Deep Learning Laboratory
Sri Ramachandra Faculty of Engineering and Technology, SRIHER, Chennai

November 2026

CONTENTS

How to Use This Guide 1

1. The Project on One Page 2
- 1.1 The idea 2
- 1.2 What was built 2
- 1.3 Headline numbers 2
- 1.4 What did not work (reported honestly) 3

2. Background Concepts from Scratch 4
- 2.1 How a game draws a frame 4
- 2.2 The G-buffer: structure without appearance 4
- 2.3 Motion vectors, warping and disocclusion 4
- 2.4 Neural networks in five minutes 5
- 2.5 The U-Net 5
- 2.6 Transformers and window attention (StructFormer) 5
- 2.7 Conditioning a network on a style: FiLM 6
- 2.8 Diffusion models, img2img and ControlNet 6
- 2.9 Losses and metrics 6
- 2.10 Training tools 7
- 2.11 Deployment: ONNX, TensorRT and precision 7
- 2.12 Research discipline used throughout 7

3. The System from Start to End 8
- 3.1 Stage 1 – The Godot game 8
- 3.2 Stage 2 – Structural export 8
- 3.3 Stage 3 – Recording and validation 9
- 3.4 Stage 4 – The diffusion teacher 9
- 3.5 Stage 5 – The four alignment gates 9
- 3.6 The confidence mask 9
- 3.7 Stage 6 – The network: StyleTemporalUNet 10
- 3.8 Training 11
- 3.9 Stages 7 and 8 – Deployment and the live loop 11

4. Results 13
- 4.1 The ablation ladder 13
- 4.2 The deployed two-style model on the game 14
- 4.3 Temporal stability and the value of history 14
- 4.4 Live performance 15
- 4.5 EPAM: the headline idea that did not help 16

5. The Project Story, Milestone by Milestone 17
- 5.1 Turning points worth knowing 17

6. Limitations 19

7. How to Run Everything 20
- 7.1 Commands 20
- 7.2 Where things are in the repository 20
- 7.3 Related deliverables 20

8. Glossary 21

9. Viva Questions and Answers 22

Appendix: All 102 Decisions 24

How to Use This Guide

This guide is written so that someone who has never seen the project, or has forgotten it, can read it once from start to end and then explain every part of StructRender: what it does, why each choice was made, how every component works, what the experiments showed, and what went wrong along the way.

- **Chapter 1** gives the whole project on one page. Read it first, and again just before a viva.
- **Chapter 2** teaches every background idea from scratch: how a game draws a frame, neural networks, U-Nets, diffusion models, losses, metrics and deployment. Skip what you already know.
- **Chapter 3** walks the pipeline from the game to the live screen, stage by stage.
- **Chapter 4** gives every result with its numbers. **Chapter 5** tells the project's history, including the bugs and the ideas that were refuted.
- **Chapters 6 to 9** cover limitations, how to run everything, a glossary and likely viva questions with answers. The appendix lists all 102 recorded decisions.

Numbers in this guide are copied from the project's own records: docs/REPORT.md, the lab report and the Decision Log in docs/StructRender_KB.md. Decisions are cited as D1 to D102.

1. The Project on One Page

A game engine keeps all gameplay truth; a small neural network only decides how each frame looks, in a learned painted style, live.

1.1 The idea

Recent research builds *generative world models* (GameNGen, Genie) that predict a game's next frame directly from earlier frames. They are impressive, but they hold no real game state, so a collected coin can come back or a wall can move. StructRender takes the opposite position: a normal game engine (Godot 4) runs the game and owns everything that is true (physics, rules, which object is which). Every frame, the engine tells a neural network *what is where*: depth, surface directions, base colours and object identities. The network paints that structure in an artistic style. Because the network never touches game state, a renderer error can never become a gameplay error.

1.2 What was built

- A playable Godot game (an arena: a player collects coins while spawned enemies chase) instrumented to export its structure every frame.
- A recorder and validator that captured **24 play sessions × 990 frames = 23,760 frames** at 512 × 288.
- An **offline diffusion teacher** (Stable Diffusion 1.5 + depth ControlNet) that paints each frame in a style, with **four alignment gates** that reject paintings that lose or invent structure, or do nothing.
- **StyleTemporalUNet**, an 8,394,915-parameter U-Net that uses the previous frame (warped by the engine's motion) and a 16-number style vector, so one model renders two styles: **oil** and **watercolour**.
- A live runtime: ONNX → TensorRT FP16 → C++ GDExtension inside Godot, with a fallback to the plain frame on any error.
- A full **ablation ladder** comparing U-Net, transformer (StructFormer), temporal input, entity awareness and a per-entity memory (EPAM), with thresholds fixed before results.

1.3 Headline numbers

Table 1: The numbers to remember

What	Result
Image quality on held-out game sessions	35.33 dB PSNR (oil), 34.79 dB (watercolour); sharpness 0.946 / 0.937 (gate 0.90)
Style learned, not ignored	colour travel 0.995 / 0.987 of the teacher's (gate 0.80); 7.3–7.8 dB better than showing the plain frame
Flicker	0.54× (oil) and 0.48× (watercolour) the teacher's own flicker
Live speed	44.6 fps oil, 44.7 fps watercolour (target 30); 4.53 ms per TensorRT FP16 step
Reliability	0 fallbacks in 3,000 live frames; 8 of 8 forced failures recovered

What	Result
Live = evaluated	live frame matches the scored model to within 1/255
Training	60 epochs, 29.2 h on an RTX 5060 Laptop GPU; best epoch 49
Tests	548 automated tests pass

1.4 What did not work (reported honestly)

- **StructFormer**, the window-attention transformer that was planned as the final renderer, never beat the plain U-Net, even with 32× more data (D49, D71).
- **EPAM**, the per-entity appearance memory that was the headline research idea, did not improve output on this data (F − F0 sharpness −0.0127, inside noise; D79).
- Moving to 768 × 432 was stopped because the teacher's look changes with resolution (D101, D102).



Figure 1: One live game frame: the native Godot render, and the same frame rendered live by StructRender in oil and in watercolour.

2. Background Concepts from Scratch

Every idea the project uses, explained so the rest of the guide reads easily.

2.1 How a game draws a frame

A game is a loop. Many times a second it (1) reads the player's input, (2) advances the simulation by a fixed step (physics, rules, enemy behaviour) and (3) draws a picture of the current state. The drawing step is **rendering**: the engine projects every 3D object through a virtual camera onto a 2D grid of pixels, works out which surface is nearest at each pixel, and shades it using materials and lights. Our game runs its physics at **60 Hz** (60 steps per second). Godot is an open-source engine; we use version 4.7.2 with its **Forward+** renderer.

The key point: the engine knows *exactly* where everything is. It does not guess. That exact knowledge is what StructRender gives to the neural network.

2.2 The G-buffer: structure without appearance

While rendering, an engine can write several per-pixel images besides the final colour. Together they are called a **G-buffer** (geometry buffer). StructRender exports four of them every frame:

- **Depth**: distance from the camera to the surface at each pixel, in metres. It tells the network the 3D shape. We feed **inverse depth** $x = 1 / (1 + d)$, which stays between 0 and 1 and maps the empty sky ($d = \infty$) to 0.
- **Normals**: the direction each surface faces, as a 3D vector (3 channels). It tells the network how surfaces are oriented, which drives shading.
- **Albedo**: the plain base colour of the material, with no lighting (3 channels).
- **Entity-ID map**: an integer per pixel saying which object (player, coin, enemy) covers it. It is used only as an *address*: to find an object's pixels or key its memory. It is never fed to the network as a number, because a recycled ID would make a new object look like the old one (D9).

So the network input is **7 channels**: inverse depth (1) + normals (3) + albedo (3), at 512 × 288 pixels.

2.3 Motion vectors, warping and disocclusion

A **motion vector** says, for each pixel, where that bit of surface was in the previous frame. Because the engine knows every object's and the camera's transform in both frames, it can compute motion *exactly* (analytically), instead of estimating it from images with an optical-flow network such as RAFT (D7).

Warping uses motion to move an image from the previous frame into the current frame's positions. If the previous frame's painted output is warped forward, it should line up with the current frame. Where it does not, something changed.

Disocclusion is a pixel that was hidden last frame and is visible now, for example the floor behind a coin that just moved. There is no valid history for it, so the network must not trust warped

information there. StructRender builds a **confidence mask** M_t from the engine's own depth and normals to mark these pixels (Section 3.6).

2.4 Neural networks in five minutes

- A **neural network** is a function with millions of adjustable numbers called **parameters** (weights). It takes an input (our 7-channel image) and produces an output (an RGB image).
- A **convolution** slides a small filter (for example 3 × 3) across the image; each output pixel is a weighted sum of a small neighbourhood. Stacking many convolutions lets the network recognise edges, shapes and textures.
- **Training** shows the network many input–target pairs. A **loss function** measures how wrong the output is. **Backpropagation** computes the **gradient**, the direction that reduces the loss, and an **optimiser** nudges every parameter that way.
- One pass over the training data is an **epoch**. A **batch** is how many samples are processed before one update. **Gradient accumulation** adds the gradients of several small batches before updating, to simulate a bigger batch when GPU memory is short.
- Data is split into **training** (learn), **validation** (choose the best checkpoint, tune) and **test** (scored once, at the end). StructRender splits by **whole play session**, so test frames never come from a session the model trained on.
- **Overfitting** is when a model memorises training data and fails on new data. A **seed** fixes the random starting point; different seeds give slightly different models, which is **run-to-run noise**.

2.5 The U-Net

A **U-Net** is an encoder–decoder network shaped like a U. The **encoder** repeatedly applies convolutions and **max pooling** (halving the image size) so features become more abstract and cover more of the image. The **bottleneck** is the smallest, most abstract representation. The **decoder** upsamples step by step back to full size. **Skip connections** copy each encoder level's features straight across to the matching decoder level, so fine detail is not lost. U-Nets are the standard network for image-to-image tasks.

Our U-Net has four encoder levels with 32, 64, 128 and 256 channels, a bottleneck of 512 channels at 18 × 32, and a mirrored decoder. Each block uses **group normalisation** (which normalises groups of channels and does not depend on batch size) and **ReLU** (a simple non-linearity: negative values become zero).

2.6 Transformers and window attention (StructFormer)

A **transformer** uses **attention**: every position looks at other positions and takes a weighted mix of their features, with weights learned from content. Attention over a whole high-resolution image is too expensive, so **window attention** (as in the Swin Transformer) restricts it to small windows. **StructFormer** was the project's transformer renderer: a convolutional stem, window-attention encoder and decoder, and a convolutional head. A variant added **entity-masked cross-attention**, where each object's pixels attend to that object's own token. StructFormer was the planned final renderer (D3), but it lost to the U-Net (Section 4.1).

2.7 Conditioning a network on a style: FiLM

To let one network produce several styles, we give it a **style vector** s: 16 learned numbers per style. **FiLM** (feature-wise linear modulation) turns s into a per-channel scale γ and shift β for each decoder layer: $h' = h \odot (1 + \gamma) + \beta$, where $\odot$ means element-wise multiplication. Choosing a different s changes the look; nothing else in the network changes. Only the **decoder** is modulated, so the encoder is **style-free**: its features are the same whichever style is shown, which lets the frame-to-frame history survive a live style switch.

Zero initialisation is used everywhere a new mechanism is added: its last layer starts at zero, so an untrained network behaves exactly like the network without the mechanism. Training then decides whether to use it.

2.8 Diffusion models, img2img and ControlNet

A **diffusion model** learns to remove noise from images. To generate, it starts from noise and denoises in many small steps. **Stable Diffusion 1.5** does this in a compressed latent space guided by a text prompt. **img2img** starts not from pure noise but from a real image with some noise added; the **strength** (0 to 1) sets how much is replaced: low strength keeps the image, high strength restyles more but may invent things.

ControlNet adds spatial control to a frozen diffusion model through a trainable side branch joined by zero-initialised layers. A **depth ControlNet** forces the generated image to follow a given depth map. StructRender feeds it the engine's own depth, so the painting keeps the real geometry. Diffusion is far too slow for live play (24 denoising steps per image), so it is used **offline only**, as a *teacher* that makes training targets (D6).

2.9 Losses and metrics

Table 2: Every loss and metric used

Name	What it measures	Better is
L1 (mean absolute error)	Average absolute difference per pixel value	Lower
MSE / PSNR	Mean squared error; PSNR = $10 \cdot \log_{10}(1 / \text{MSE})$ in decibels	Higher PSNR
SSIM	Similarity of local brightness, contrast and structure (1 = identical)	Higher
LPIPS	Distance between deep (AlexNet) features; tracks human judgement	Lower
Sharpness ratio	High-frequency (Sobel gradient) energy of output ÷ target; 0.90 ≈ a 1-pixel blur	Close to 1; gate ≥ 0.90
Colour (histogram) travel	How far the model moves the colour histogram from the plain frame, relative to the teacher	Gate ≥ 0.80
Temporal error (flicker)	Difference between a frame and its warped predecessor on confident pixels	Lower, read with sharpness

Two subtleties matter. First, pixel losses like L1 reward **blur**: when the target is uncertain, the average is the safest guess (the *perception–distortion trade-off*). That is why every result is paired with the **sharpness ratio**. Second, a frozen or blurred output has tiny temporal error, so temporal error is never reported without sharpness.

The training loss is L = L1 + 0.5 · LPIPS + 0.5 · (masked temporal term). The temporal term compares each predicted frame with the previous predicted frame warped forward, only where the confidence mask trusts history.

2.10 Training tools

- **AdamW**: an adaptive optimiser with decoupled weight decay (a pull of weights toward zero that reduces overfitting). Learning rate 2×10^{-4}, weight decay 10^{-4}.
- **Cosine schedule**: the learning rate decreases smoothly along a cosine curve over training.
- **bfloat16 mixed precision**: computing in 16-bit floats where safe, to save memory and time on the GPU.
- **Early stopping**: stop if validation stops improving for a set number of checks (patience 6 evaluations); it was not triggered in the final run.
- **Checkpoint selection**: keep the epoch with the lowest validation L1 (epoch 49 of 60).

2.11 Deployment: ONNX, TensorRT and precision

- **ONNX** (Open Neural Network Exchange) is a standard file format for a trained network's computation graph, so it can leave PyTorch.
- **TensorRT** is NVIDIA's inference optimiser. It compiles an ONNX graph into an **engine** tuned for one specific GPU and driver, which is why engines are built on the target laptop (D16).
- **Precision**: FP32 (32-bit floats) is exact; TF32 is a faster, slightly less precise mode for FP32 maths on NVIDIA GPUs; FP16 (16-bit) is fastest. StructRender ships FP16 after checking it changes PSNR by under 0.05 dB.
- **Latency** is the time for one network step (4.53 ms for FP16). **Frame rate** (fps) is for the whole loop: capture, motion, inference and display (44.6 fps).
- A **GDExtension** is Godot's way to call native C++ code from game scripts. Our C++ runtime loads the TensorRT engines and is called from GDScript every frame.

2.12 Research discipline used throughout

- **Pre-registration**: every pass/fail threshold was written into the evaluator code or the Decision Log *before* the result existed, so no goalpost could move after seeing numbers.
- **Noise threshold**: a difference between two single training runs is claimed only if it exceeds $2\sqrt{2} \times$ the relative seed-to-seed standard deviation (from three seeds).
- **Sealed test sets**: test sessions were chosen by position before any model existed and scored once, at the end.
- **Ablation ladder**: models that differ by one idea each, at the same parameter count (0.9–1.1×), same data, loss and stopping rule, scored by one script.
- **Negative results are kept**: when a hypothesis failed, the result was recorded, not tuned away.

3. The System from Start to End

Eight stages: Godot game → export → record → teacher → gates → train → deploy → live.

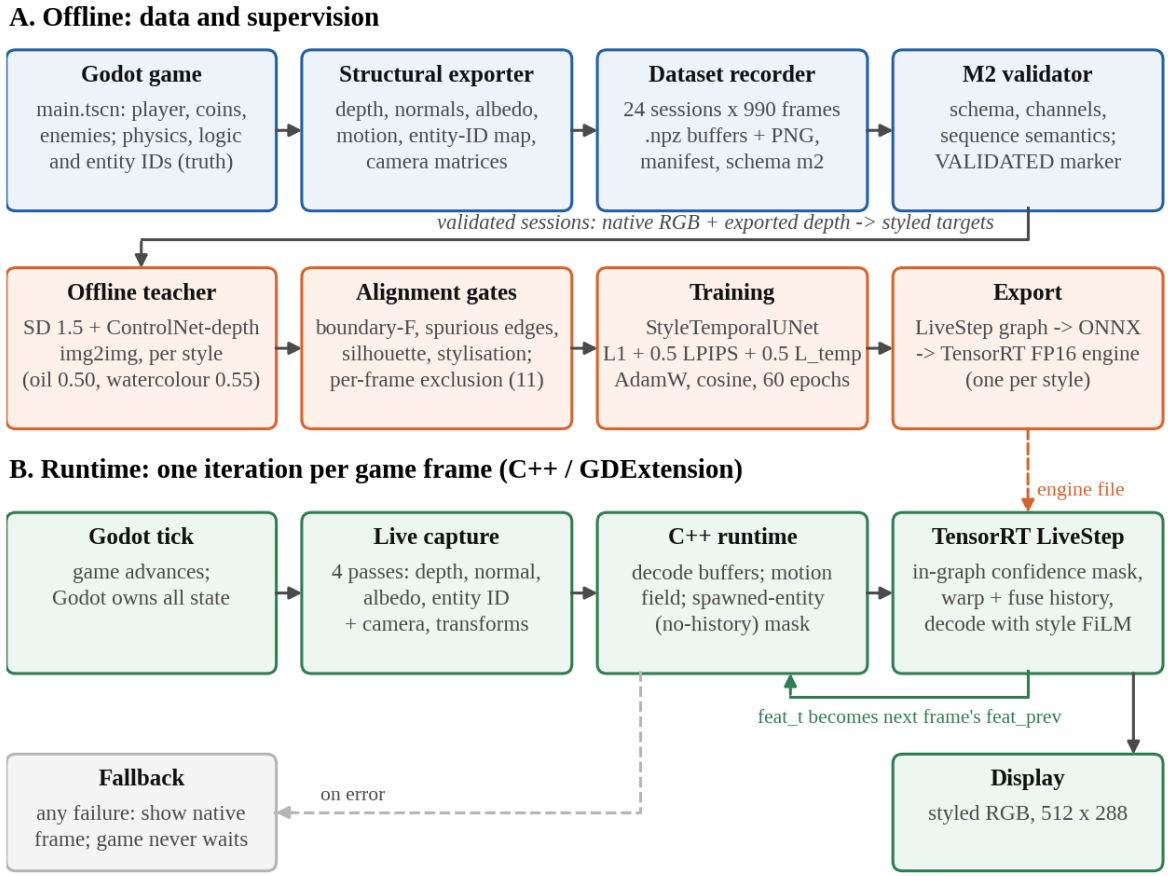


Figure 2: The whole system: the offline part (top) produces a validated dataset and a trained model; the runtime loop (bottom) runs once per game frame inside Godot.

3.1 Stage 1 – The Godot game

The game (game_project/, scene main.tscn) is a small arena: the player collects coins while enemies spawn and chase. Physics runs at 60 Hz. The engine is the single source of truth for physics, rules and entity identity (D1, D2). The live controls are WASD or arrow keys plus Space; in the live renderer, key **1** selects oil, **2** watercolour and **0** the native look.

3.2 Stage 2 – Structural export

The exporter renders the scene several more times into an off-screen viewport, each time replacing every material with a simple unshaded shader that writes one quantity: depth, normals, albedo or a packed entity ID. Each image is read back. Motion vectors are computed on the CPU from the stored camera and object transforms (D26). This *override-shader* approach was chosen because Godot gives no stable script access to its internal G-buffer, and because each pass can be checked directly.

3.3 Stage 3 – Recording and validation

A scripted recorder drives the game through its own input actions. For each frame it advances exactly **one physics step** and pauses the scene while all passes render, so every buffer shows the same instant. A validator then rejects sessions with schema errors, missing channels, non-finite values or inconsistent object lifecycles. One seed-scene session was rejected for non-finite motion at frame 616.

Table 3: The two datasets

	Game dataset (deployed model)	**Seed-scene dataset (ablation ladder)**
Source	The playable game, scripted play	An arena built in code
Sessions	24 recorded, 24 valid	30 recorded, 29 valid
Frames	990 per session; 23,760 total	990 per session; 28,710 total
Split (sessions)	18 / 3 / 3 (train / val / test)	21 / 4 / 4
Split (frames)	17,820 / 2,970 / 2,970	20,790 / 3,960 / 3,960
Styles	oil 0.50; bold watercolour 0.55	oil 0.50

3.4 Stage 4 – The diffusion teacher

No painted versions of game frames exist, so the project makes them. For each frame and style, Stable Diffusion 1.5 img2img with a depth ControlNet restyles the native frame, guided by the engine's depth. Settings: 24 steps, guidance 7.5, ControlNet scale 1.6; strength 0.50 for *oil impasto* (seed 20260906) and 0.55 for *bold watercolour* (seed 20260930). Generation is seed-pinned and cached by content, so it is reproducible. The teacher never runs during play.

3.5 Stage 5 – The four alignment gates

A painting is only accepted as a training target if it passes four gates, measured against the engine's own structure, with thresholds fixed before the game data existed:

Table 4: Alignment gates

Gate	**Question it asks**	**Pass if**
1. Boundary F-measure	Are the real structural edges still there (2-pixel tolerance)?	≥ 0.60
2. Spurious-edge ratio	Did the painting invent strong edges with no structure behind them?	≤ 0.35
3. Silhouette support	Does every object of ≥ 200 visible pixels keep its outline?	≥ 0.60
4. Stylisation	Did the painting actually change the colours (histogram L1 from the native frame)?	≥ 0.15

Gate 4 exists because gates 1–3 are passed perfectly by a teacher that changes nothing: the untouched native frame scored boundary F 0.976 and spurious 0.000, better than any real painting (D32). A frame failing any gate in either style is excluded from both styles, along with every training sample that touches it: **11 of 23,760 frames** (D94).

3.6 The confidence mask

For each pixel, the mask compares the current depth and normal with the previous frame's, warped by motion:

$$M_t = \exp(-\kappa_D \cdot e_D - \kappa_N \cdot e_N) \cdot H_t$$

Here e_D is the depth disagreement, $e_N = 1 - (\text{normal} \cdot \text{warped normal})$, $\kappa_D = \ln 2 / 0.175$ m and $\kappa_N = \ln 2 / (1 - \cos 30°)$. So a depth step of 0.175 m, or a surface turning by 30°, halves the confidence. H_t is 0 where there is no valid history at all: the first frame, motion from off-screen, the background, and newly spawned objects (found from the entity-ID map). The 0.175 m scale was calibrated on the data as the best separation between depth changes within a surface and across an object's edge.

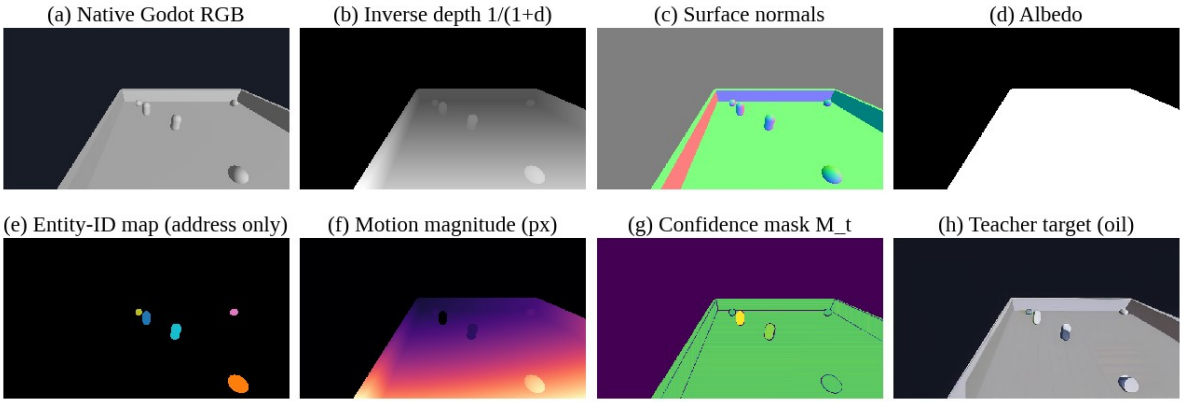


Figure 3: One sealed test frame: (a) native render, (b)–(d) the network inputs, (e) entity IDs (address only), (f) motion magnitude, (g) confidence mask, (h) oil teacher target.

3.7 Stage 6 – The network: StyleTemporalUNet

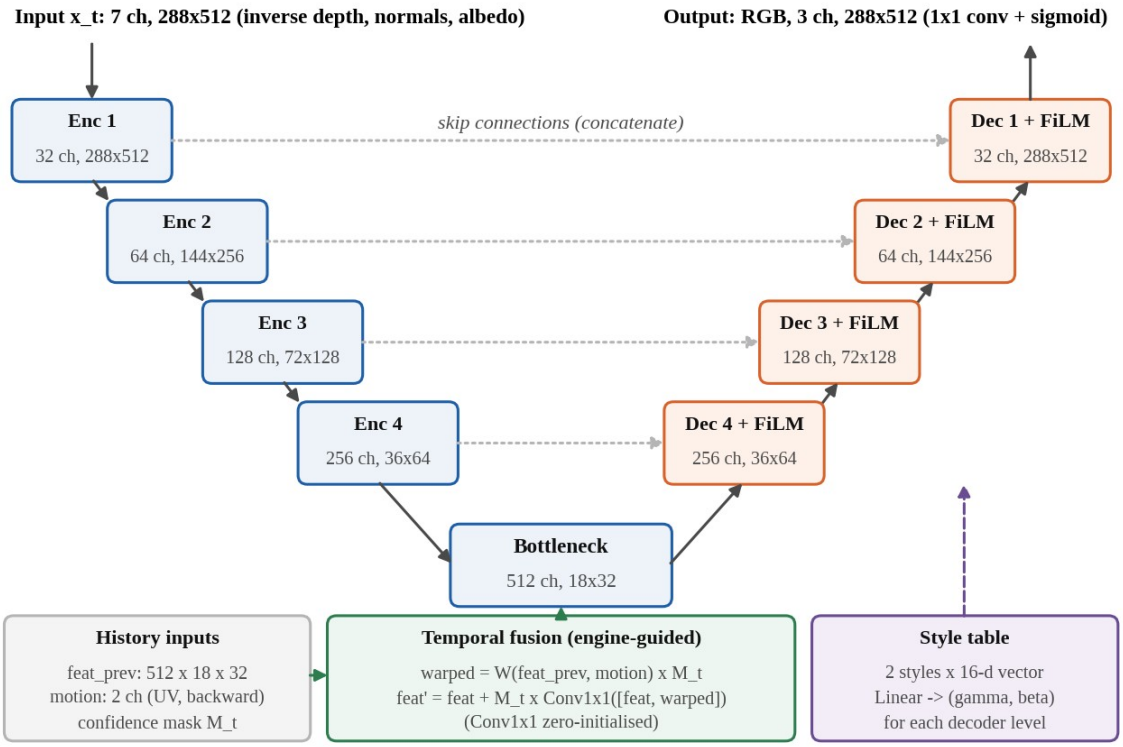


Figure 4: StyleTemporalUNet: style-free encoder, temporal fusion at the bottleneck, FiLM-modulated decoder.

- **Encoder**: four levels (32, 64, 128, 256 channels), each two 3 × 3 convolutions with group norm and ReLU, then 2 × 2 max pooling; bottleneck 512 × 18 × 32.

- **Temporal fusion**: last frame's bottleneck features are warped by engine motion, multiplied by the mask, and fused with a zero-initialised 1 × 1 convolution: $F' = F + M \odot \mathrm{Conv}([F, M \odot W(F_{prev})])$. Where M = 0, history is ignored.
- **Decoder**: four upsampling levels with skip connections, each modulated by FiLM from the 16-number style vector.
- **Head**: a 1 × 1 convolution and a sigmoid give RGB in [0, 1].
- **Size**: 8,394,915 parameters (encoder 1,175,328; bottleneck 3,542,016; temporal fusion 524,800; decoder 3,136,320; FiLM 16,320 + 32; head 99).

3.8 Training

Table 5: Training configuration of the deployed model

Item	Setting
Samples	35,526 training samples (both styles); 5,936 validation frames
Each sample	three consecutive frames, so the model learns to use the previous frame
Loss	L1 + 0.5 · LPIPS (AlexNet) + 0.5 · masked temporal term
Optimiser	AdamW, learning rate 2×10^{-4}, weight decay 10^{-4}, cosine schedule
Batch	4 × gradient accumulation 2 (effective 8); bfloat16
Length	60 epochs, validation every 2; 29.2 hours; best epoch 49 (validation L1 0.007435)
Hardware	ASUS ROG Strix G16 (2025): Ryzen 9 8940HX, RTX 5060 Laptop GPU 8 GB, 30 GiB RAM

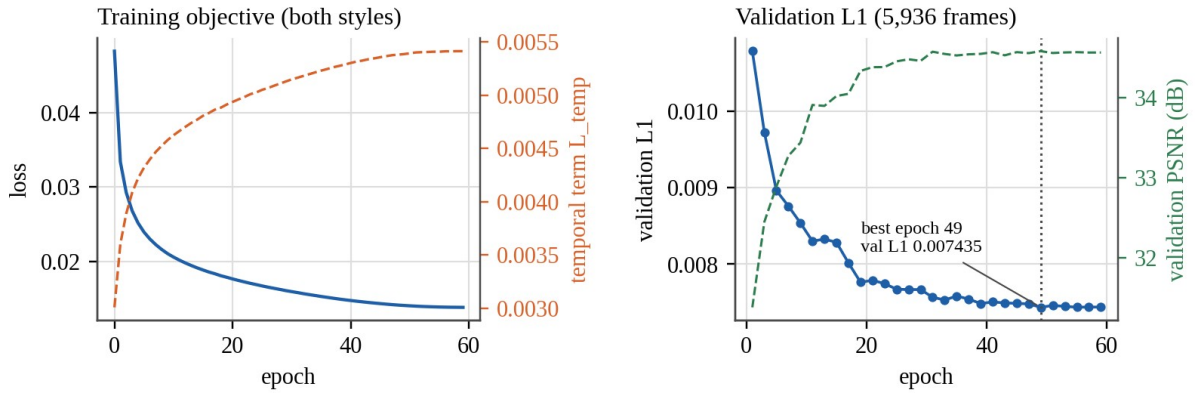


Figure 5: Training of the deployed model. The temporal term rises as the output gains texture, which is why flicker is always read together with sharpness.

3.9 Stages 7 and 8 – Deployment and the live loop

One step of the model, including the confidence mask, is exported as a graph called *LiveStep*: six inputs (current structure, previous structure, previous features, motion, new-object mask, motion validity) and two outputs (the frame, and this frame's features for next time). It is compiled to one TensorRT FP16 engine per style, with the style fixed inside each engine. Each game frame:

1. Godot steps physics and captures four passes (depth, normals, albedo, entity ID).
2. The C++ GDExtension computes motion and the mask of newly spawned objects, ported line by line from the training code.
3. TensorRT runs LiveStep for the selected style with the previous frame's features.

4. The frame is displayed and its features are kept for the next frame. A style switch keeps the history, because the encoder is style-free.
5. If anything fails, the native frame is shown instead and the game never waits.

Table 6: Proof that the live frame is the evaluated model

Link checked with identical inputs	**Maximum difference**
Exported LiveStep vs the Python evaluator	2.4×10^{-6}
TensorRT strict FP32 vs PyTorch (frame / features)	1.6×10^{-6} / 7.3×10^{-6}
C++ motion vs the engine's own motion, 120 frames	7.5×10^{-8}
Live game frame vs the scored model	1/255 (rounding); 0 pixels above
Spawned enemy: no-history pixels, C++ vs engine	998 = 998 pixels

4. Results

The U-Net with temporal input wins; the deployed two-style model passes every gate and runs at 44.6 fps.

4.1 The ablation ladder

Every rung was scored on the same 3,960 sealed test frames of the seed scene by one script; temporal error uses 501 moving frame pairs.

Table 7: Ablation ladder

Rung	PSNR	SSIM	LPIPS	Sharpness	Temporal error	Params
A U-Net	34.04	0.9747	0.0459	0.912	0.003175	7.85 M
B U-Net + temporal	33.64	0.9734	0.0489	0.897	0.002692 (−15.2 %)	8.38 M
C StructFormer	33.86	0.9728	0.0466	0.853	0.004191	7.78 M
D StructFormer + temporal	33.56	0.9727	0.0485	0.858	0.003741 (−10.7 %)	8.24 M
E StructFormer + entity	33.79	0.9726	0.0463	0.884	0.004172	7.05 M
N raw-ID control	33.86	0.9732	0.0459	0.880	–	7.78 M
Teacher / native engine	–	–	–	–	0.006265 / 0.000241	–

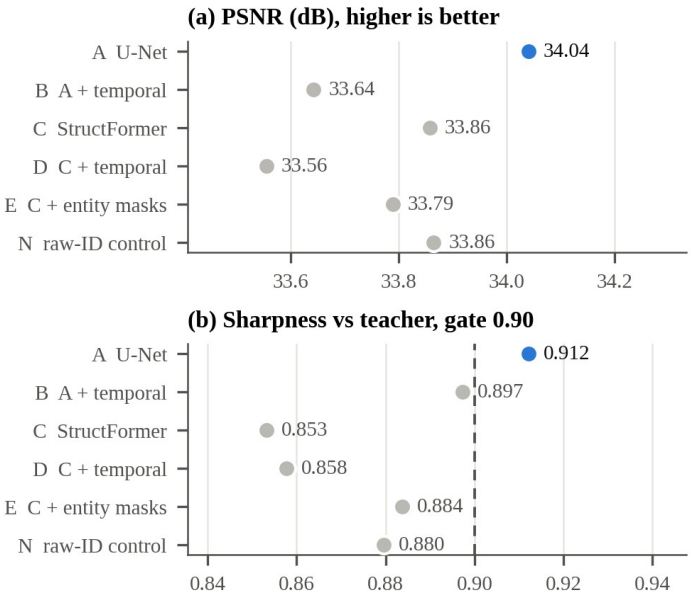


Figure 6: Ablation ladder: the plain U-Net (A) leads on PSNR and is the only rung above the 0.90 sharpness gate.

- **The U-Net (A) is best on every image metric** and the only rung above the sharpness gate. StructFormer's sharpness was 0.833, 0.886 and 0.851 at 644, 10,890 and 20,790 training frames, against a pre-recorded projection of 0.898: more data did not close the gap (D71).
- **Temporal input cuts flicker on both backbones**: −15.2 % (B vs A) and −10.7 % (D vs C), far beyond the pre-registered 3.85 % threshold, at a cost of 1.8–3.9 % in per-frame L1. Flicker is the more visible defect in play, so **B was taken to the game**.
- **Entity awareness helps a little**: E is +0.015 sharper than C over three seeds (2.5σ). But the naive control N, which just feeds the raw ID, gets about 87 % of that gain: with only two or three objects, exposing the IDs is enough (D89).

4.2 The deployed two-style model on the game

Table 8: Game test sessions, 2,968 frames per style

Model, style	PSNR	SSIM	LPIPS	Sharpness	Travel	Flicker vs teacher
Two-style B, oil	35.33	0.9807	0.0225	0.946	0.995	0.54×
Two-style B, watercolour	34.79	0.9786	0.0243	0.937	0.987	0.48×
Oil-only B (reference)	35.43	0.9814	0.0216	0.946	0.980	0.54×
Plain frame (do nothing)	28.06 / 26.96	–	–	–	–	–

- Both styles pass the sharpness and travel gates and beat doing nothing by 7.3–7.8 dB.
- Sharing one model between two styles costs oil 0.10 dB, which is not claimed as real with one run per model.
- The switch is real, not a blend: each style's output is about 4× closer to its own targets than to the other style's, reproducing 93 % of the difference between the two teachers.
- The styles are mild because the game is grey and plainly lit, and stronger teacher settings fail the gates.

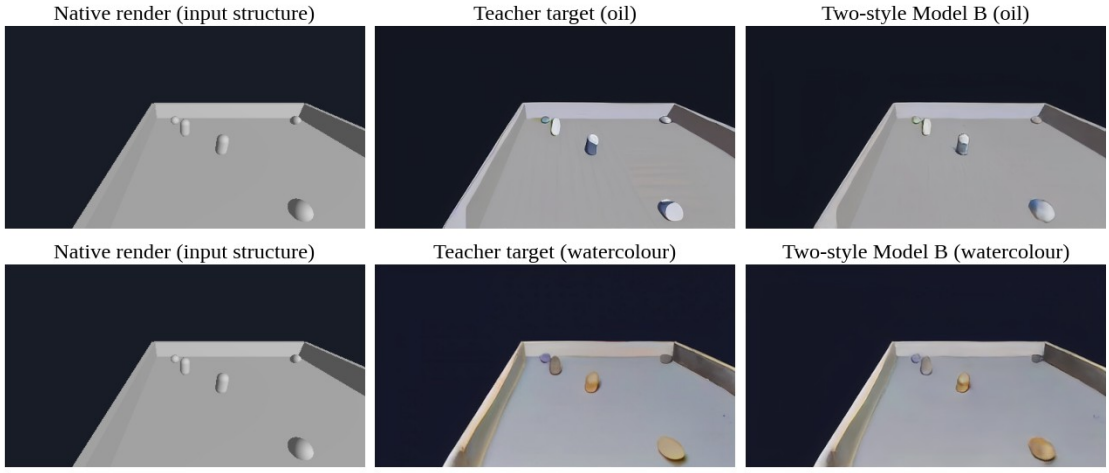


Figure 7: A sealed test frame through exactly the evaluated path: native input, teacher target and model output for both styles.

4.3 Temporal stability and the value of history

The renderer flickers about half as much as its own teacher (0.54× oil, 0.48× watercolour, on 2,451 moving frame pairs), but still about 57× the native engine: the teacher's flicker is the ceiling. History also improves image quality: run cold every frame, the same model drops from 35.33 to 33.58 dB (oil) and from 34.79 to 32.82 dB (watercolour), which is why the live runtime feeds features back every frame.

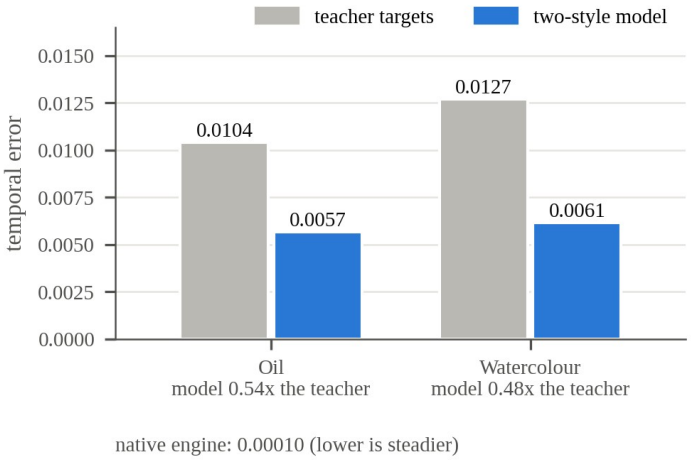


Figure 8: Flicker of the teacher's targets and of the two-style model on the game's test sessions.

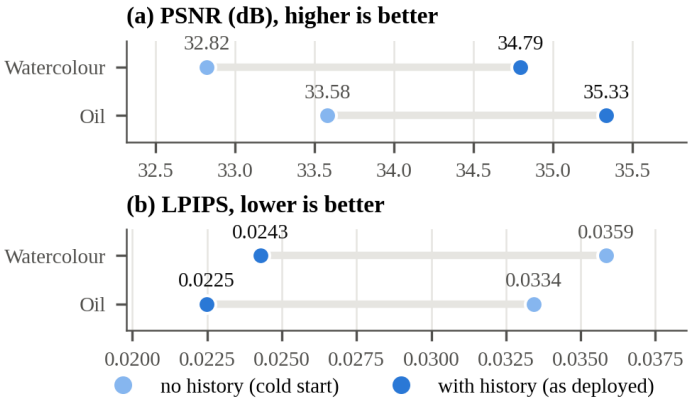


Figure 9: The same model with and without its one frame of engine-warped history.

4.4 Live performance

Table 9: Live measurements on the playable game

Criterion	Result	Verdict
About 30 fps at 512 × 288	44.6 fps oil, 44.7 fps watercolour (FP16)	Met
Network alone ≤ about 20 ms	4.53 ms FP16; 6.23 ms strict FP32; 9.37 ms with TF32	Met
Stable loop	3,000 frames, 0 fallbacks; median 22.4 vs 22.3 ms (first vs second half); p99 25.2 ms	Met
Failure does not break play	8 of 8 forced failures fell back to the native frame; play continued	Met
Manual play session (23 Oct 2026)	2,059 styled frames, 40.1 fps oil and 38.0 fps watercolour, 0 fallbacks	_

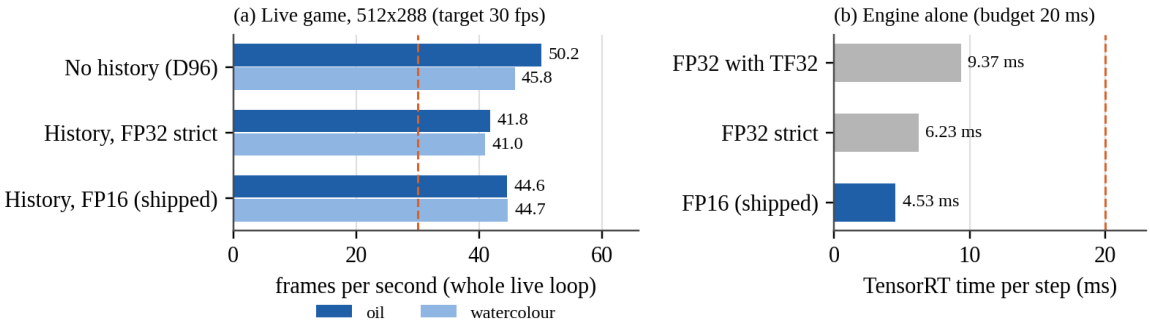


Figure 10: Frame rate of the whole live loop, and TensorRT time per step alone.

4.5 EPAM: the headline idea that did not help

EPAM (Engine-Keyed Persistent Appearance Memory) keeps a memory slot per object, keyed by the engine's entity identity. While the object is visible enough, its descriptor is blended into the slot by an exponential moving average (decay $\lambda = 0.9$); the slot is frozen while the object is out of view, deleted when it is destroyed, and cleared on a style change. The goal: an object should look the same when it reappears.

The test was clean. Model F (memory on) and its twin F0 ($\lambda = 0$, memory off) share code, data, schedule and selection rule (D78). The pass threshold, 0.0206 sharpness, was written before the run. Result: F − F0 = **−0.0127** (0.8829 vs 0.8956) on 3,912 test clips: inside noise and in the wrong direction (D79). Three further measurements agree that the data offers memory nothing to recover: appearance correlation across an occlusion gap is at most 0.037 beyond 30 ticks (D73), the reappearance metric does not degrade with time unseen (D53), and anchoring the teacher's noise per object does not create a persistent look (D81). The likeliest reason is the supervision: the diffusion teacher re-invents each object's look every frame, so there is no stable appearance in the targets to remember. The lifecycle rules themselves are met and covered by 17 tests.

5. The Project Story, Milestone by Milestone

Eleven milestones, each with exit criteria fixed in advance; failures were recorded as findings.

Table 10: Milestone scorecard

Milestone	Verdict	Key decisions
M0 Repository and minimal Godot game	Complete	D22–D24
M1 Structural export	Complete	D23–D26
M2 Dataset recorder and validator	Complete	D27–D30
M3 Offline teacher and alignment gates	Complete	D31–D34, D40
M4 CNN / U-Net baseline	Passes at every dataset size	D35, D41, D46, D71
M5 StructFormer baseline	Fails: below the U-Net at 644, 10,890 and 20,790 frames	D49, D71
M6 Temporal reprojection	Passes (−11.7 %); replicated (−10.7 %)	D67, D85
M7 Structured entity awareness	Complete	D74
M8 EPAM	Criteria 1–3 met; criterion 4 (improves output) not met	D78, D79
M9 Full evaluation (ablation ladder)	Complete	D83, D85, D89
M10 ONNX → TensorRT → C++ → Godot	Passes all exit criteria	D84, D86, D96–D99

5.1 Turning points worth knowing

- **The design (D1–D21).** The engine is the source of truth; diffusion is an offline teacher; engine motion replaces optical flow; entity IDs are addresses; EPAM is the headline idea and is session-local; TensorRT engines are built on the target laptop; FP16 before FP8.
- **A fourth gate (D32).** Calibration showed that a teacher that does nothing passes the three structural gates perfectly, so the stylisation gate was added. The first teacher setting also failed on moving-camera frames until ControlNet conditioning was strengthened.
- **The scene limited the style (D36–D40).** The stylisation gate proved too permissive; the cause was the plain scene geometry, not the teacher. The scene was enriched, which rescued the failing configuration, and oil_impasto became the committed style.
- **Measuring noise properly (D46).** Three seeds showed that two runs badly understate run-to-run variance, and that M4's sharpness gate is seed-fragile. All later margins were set from measured noise.
- **StructFormer refuted (D49, D71).** The transformer lost to the CNN at equal size, and at 32× the data its sharpness fell instead of rising, refuting the scaling hypothesis.
- **M6 first failed, then passed (D57, D67).** Temporal reprojection's gain was first inside StructFormer's own seed noise; with the correctly measured margin on the larger ladder, it passed.
- **A tempting fix made things worse (D60).** Conditioning the teacher on its own previous painting made flicker 11× worse (378× the engine), most likely because it re-stylised already-styled frames again and again.

- **A silent bug found and fixed (D66, D80).** Model B's temporal fusion looked inert. The real cause: B had gone through the single-frame trainer, so the fusion never received any gradient. The trainer was fixed and guarded; once trained correctly, temporal input cut flicker on both backbones (D85).
- **A TensorRT failure diagnosed (D84).** Model A would not build in TensorRT. Changing one thing at a time showed that GroupNorm, exported as ONNX InstanceNormalization, triggered a compiler failure; rewriting GroupNorm as plain maths fixed it, verified to 4.26×10^{-6}.
- **From the test scene to the real game (D87–D95).** The game itself was recorded (23,760 frames). Oil passed the gates; the original watercolour failed and a bolder watercolour passed at 0.55. Eleven failing frames were excluded instead of dropping whole sessions. The two-style model was built and scored.
- **Live in the game (D96–D99).** Style switching first ran without history (about 2 dB worse); the LiveStep graph then brought the scored model's history into the game. FP16 passed its pre-set tolerance and was shipped; failure injection, stability runs and the last open checks closed M10.
- **Higher resolution stopped (D100–D102).** A 768 × 432 dataset was pre-registered, but the teacher's look changes with resolution, and re-tuning strength could not recover the committed look, so the bulk run was stopped.

6. Limitations

- **One scene, one game.** Nothing here is evidence about other games or art styles.
- **The teacher's flicker is the ceiling.** The renderer halves it but stays about 57× above the engine.
- **Single training runs on the game.** Small differences between game models (such as 0.1 dB) are not claimed.
- **Mild styles at low resolution.** 512 × 288 and strength about 0.5, set by what passes the gates on a plain grey game.
- **Scripted, not human, recording**; the 11 excluded frames are the hardest views and are slightly under-represented.
- **Free play is not lockstep.** A physics tick can occasionally land mid-capture during free play; every equivalence check was run in lockstep.
- **EPAM and structured entity use are untested where they should matter**: many objects, recycled IDs and a temporally consistent teacher.

7. How to Run Everything

7.1 Commands

All Python commands run from the repository root in the dl conda environment (Python 3.10.21, PyTorch 2.13.0).

Table 11: Main commands

Purpose	Command
Run all tests	python -m pytest -q (or ./scripts/test.sh)
Score the game model on the sealed test sessions	python -m scripts.eval_game checkpoints/game_b_two_style/best.pt configs/data/game_split_v1.json oil_impasto eval/reports/game_b_two_style_test_oil.json configs/data/game_frame_exclusions_v1.json
Build the live TensorRT engines for this GPU	python -m scripts.m10_live_temporal_engines, then python -m scripts.m10_fp16_live
Play the game live	set SR_ENGINE_OIL and SR_ENGINE_WATERCOLOR to the FP16 engines in artifacts/engines/, then godot --path game_project -s res://tests/runtime/live_game.gd; keys 1 oil, 2 watercolour, 0 native
Remake the report figures and demo video	python -m scripts.make_report_figures; python -m scripts.make_demo_video

Trained weights are published as GitHub releases game-v1 (the game models) and ladder-v3 (ladder rungs A, C and E). No training command is listed here: training is only started deliberately.

7.2 Where things are in the repository

Table 12: Repository map

Folder	Contents
game_project/	The Godot game, structural exporter, recorder, live-runtime scripts
data_pipeline/	Recording, validation, teacher generation and the alignment gates
renderer/	Models (U-Net, StructFormer, temporal, EPAM), losses, training
inference/	ONNX export, TensorRT build, the C++ runtime and GDExtension
eval/	Metrics, protocols, reports (eval/reports/) and logs (eval/logs/)
configs/	Data splits, frame exclusions, training and teacher configurations
scripts/	One-command entry points: evaluate, build engines, make figures
docs/	Knowledge base and Decision Log, final report, lab report, paper, presentation, this guide
checkpoints/, artifacts/	Model weights and built engines (not in git)

7.3 Related deliverables

- **Lab report**: docs/lab_report/StructRender_Lab_Report.pdf (33 pages, SRIHER format).
- **Conference paper draft**: docs/paper/ (IEEE format; Word, PDF and an Overleaf project).
- **Presentation**: docs/presentation/StructRender_Lab_Presentation.pptx (19 slides).
- **Demo video**: eval/reports/demo/structrender_demo.mp4 (native vs live, switching styles).
- **Separate project**: the Spider-Man web-swinging game at structrender.vercel.app has 16 hand-made shader themes. It does not run the StructRender model; always present it that way.

8. Glossary

Table 13: Terms used in the project

Term	Meaning
Ablation	Removing or adding one component at a time to measure its effect
Albedo	A surface's base colour without lighting
Alignment gate	A pass/fail test that a teacher painting keeps the engine's structure and is really stylised
bfloat16	A 16-bit number format used to train faster with less memory
Confidence mask (M_t)	Per-pixel weight saying how far the previous frame can be trusted
ControlNet	A side network that makes a diffusion model follow a control image such as depth
Diffusion model	A generator that creates images by removing noise step by step
Disocclusion	A pixel newly revealed this frame, with no valid history
Entity ID	The engine's identity for an object; used only as an address
EPAM	Engine-Keyed Persistent Appearance Memory: a per-object memory of appearance
FiLM	Feature-wise linear modulation: a learned scale and shift per channel, from a conditioning vector
FP16 / FP32 / TF32	Floating-point precisions: 16-bit, 32-bit, and NVIDIA's fast 32-bit mode
G-buffer	Per-pixel geometric images (depth, normals, albedo) produced while rendering
GDExtension	Godot's interface for calling native C++ code
LiveStep	The exported one-frame step of the model used in the game
LPIPS	A perceptual distance computed from deep network features
Motion vector	Where each pixel's surface was in the previous frame
ONNX	A standard file format for neural-network graphs
Pre-registration	Fixing a test's threshold before seeing its result
PSNR / SSIM	Pixel-accuracy and structural-similarity image metrics
Sharpness ratio	Output's high-frequency energy relative to the target's; guards against blur
StructFormer	The project's window-attention transformer renderer
StyleTemporalUNet	The deployed U-Net with temporal fusion and FiLM style conditioning
Teacher	The offline diffusion model that makes the training targets
TensorRT	NVIDIA's compiler that turns a network into a fast GPU engine
Warp (W)	Moving an image or features by motion vectors into the current frame

9. Viva Questions and Answers

Q. What is StructRender in one sentence?

A renderer in which the Godot engine keeps all gameplay truth and a small neural network paints each frame from the engine's structure in a learned style, live at about 45 fps.

Q. Why not just use a generative world model like GameNGen?

Those models predict pixels and hold no authoritative state, so world facts can drift. We keep the engine in charge, so the network can only change how things look, never what happens.

Q. Where do the training targets come from if no painted game frames exist?

An offline teacher: Stable Diffusion 1.5 img2img with a depth ControlNet driven by the engine's own depth. Each target must pass four alignment gates before it is used.

Q. Why four gates and not three?

The three structural gates are passed perfectly by a teacher that changes nothing (the native frame scored boundary F 0.976 and spurious 0.000). The stylisation gate rejects that passthrough.

Q. Why is the entity ID not an input channel?

It is an arbitrary label. As a number, a recycled ID would make a new object look like the old one. We use IDs only as addresses for masks and memory.

Q. What does the confidence mask do?

It marks pixels where the previous frame can be trusted, using depth and normal agreement after warping, and zeroes background, the first frame, off-screen motion and newly spawned objects. History is ignored where the mask is zero, both in training and live.

Q. Why a U-Net and not the transformer?

We tested both at equal size. The U-Net won on every image metric and was the only model above the sharpness gate; the transformer did not catch up even with 32× more data.

Q. How does one model produce two styles?

A learned 16-number style vector drives FiLM scale and shift in the decoder. The encoder never sees the style, so its features, and the frame history, carry across a live style switch.

Q. How do you know the live game shows the model you evaluated?

Every link was checked with identical inputs: export vs evaluator (2.4×10^{-6}), TensorRT vs PyTorch (1.6×10^{-6}), C++ motion vs engine motion (7.5×10^{-8}), and the live frame vs the scored model (within 1/255).

Q. Why FP16?

It is 27 % faster than strict FP32 (4.53 vs 6.23 ms) and passed a tolerance fixed before measurement: mean PSNR change 0.044 dB, under 0.05.

Q. What happens if the renderer fails mid-game?

The native frame is shown and the game continues; 8 of 8 forced failures recovered, and 3,000 live frames had 0 fallbacks.

Q. Why is the output still flickery compared with the plain game?

The teacher paints each frame independently, so its targets flicker. The model halves the teacher's flicker, but cannot remove flicker that is in its own training targets.

Q. Did EPAM work?

No. The memory model was 0.0127 less sharp than its identical no-memory twin, inside noise. Other measurements show the targets carry no stable per-object appearance for a memory to recover; a temporally consistent teacher would be needed.

Q. What was the most surprising bug?

Model B's temporal fusion looked useless, but it had been trained by the single-frame trainer, so the fusion never received a gradient. Once trained correctly, temporal input cut flicker by 15.2 %.

Q. Why are the styles so subtle?

The game is grey and plainly lit, and stronger teacher settings start inventing edges and fail the alignment gates, so the accepted targets are mild.

Q. Why not higher resolution?

At 768 × 432 the teacher's look changes, and re-tuning could not reproduce the committed look, so the run was stopped before spending compute.

Q. What would you do next?

A temporally consistent teacher; re-test EPAM in scenes with many objects and recycled IDs; higher resolution with a matched look; and evaluation on more games and on human play.

Q. Are the Spider-Man screenshots your model's output?

No. They come from a separate browser game with 16 hand-made shader themes. StructRender has two trained styles and runs on the Godot game.

Appendix: All 102 Decisions

The Decision Log in docs/StructRender_KB.md records every design choice, deviation, result and correction, in order. Titles are reproduced here as an index.

Table 14: Decision Log index

No.	Decision
D1	Deterministic Engine Over Pure Generative World Simulation
D2	Godot Is the Source of Truth
D3	StructFormer Replaces GAN-First as the Final Renderer
D4	GAN Becomes Optional Training Supervision
D5	One-Step DiT Is Not a Required Deliverable
D6	Heavy Diffusion Is an Offline Teacher
D7	Engine Motion Replaces RAFT in Production
D8	Motion Has an Explicit Operation, Not a Raw Channel Role
D9	Entity IDs Are Addresses, Not Continuous Features
D10	EPAM Is the Headline Contribution
D11	EPAM Is Session-Local
D12	Style Change Clears EPAM in v1
D13	No Arbitrary EPAM Eviction in the Core Benchmark
D14	Window Attention Over Global Full-Resolution Attention
D15	Runtime Style Encoding Is Cached
D16	TensorRT Engine Is Built on the Target Laptop
D17	C++ Owns the Final Runtime Orchestration
D18	FP16 Before FP8
D19	Train in Stages and Resume Checkpoints
D20	Paid GPU Budget Is Reserved for High-Value Runs
D21	Entity Token Overflow Is Resolved by Visibility Ranking, Not Arbitrary Order
D22	M0 Logging Autoload Is Named GameLog, Not Logger (2026-09-03)
D23	Structural-Export Render Path Is Pinned to Forward+ (2026-09-03)
D24	Test / Instrumentation Autoloads May Ship in game_project, Guarded (2026-09-03)
D25	Motion Vector Convention Pinned (2026-09-03)
D26	M1 Structural Export Is Override-Shader Re-Render, Not Native G-Buffer Readback (2026-09-03)
D27	M2 Dataset Storage Format and Schema Extensions (2026-09-03)
D28	M2 Seed Reappearance Ladder Is a Training Fixture, Distinct from the Eval Headline Ladder (2026-09-03)
D29	M2 Dataset Validator Acceptance Rules (2026-09-03)
D30	M2 Loose Ends — Validator Tolerances, Native-RGB Capture Settings, --limit Mode (2026-09-03)
D31	M3 Teacher Method, Style Scope, and Sidecar Target Storage (2026-09-05)
D32	M3 Alignment Gates — Four, Not Three; Calibrated Thresholds (2026-09-05)
D33	Seed Sessions Must Carry Visual Diversity; Target Cache Is Content-Addressed (2026-09-05)
D34	M3 Review Follow-ups — Measurement Limits, Enforced Rules, Code Location (2026-09-05)
D35	M4 Baseline — Input Channels, Sequence-Holdout Split, Pre-Fixed Overfit Thresholds (2026-09-05)
D36	M3's Stylisation Gate Is More Permissive Than It Looked — Strengthen the Style Before M5 (2026-09-06)
D37	Scene Geometry, Not the Teacher Pipeline, Is What Limits Style Strength (2026-09-06)
D38	Structural Albedo Never Reached the Render — Scene Enrichment Scope (2026-09-06)
D39	The Enriched Scene Rescues the Config That Failed — Decision 37's Diagnosis Confirmed (2026-09-06)
D40	oil_impasto Is the Committed Style; gouache Retired (2026-09-06)
D41	M4 Adds the Core Perceptual Loss; Detection Floors Re-measured (2026-09-06)
D42	Overfit Gate Uses the Configured Objective; Reports Must Self-Identify (2026-09-06)

No.	Decision
D43	The Scene Enrichment Traded Hallucination Sensitivity for Translation Sensitivity (2026-09-06)
D44	Pre-Commit Review — A Pre-Registered Threshold Changed Units; Two Guards Did Not Guard (2026-09-08)
D45	M5 Export Probe — §12's Attention Risk Did Not Materialise; Four Written Rules Did Not Survive Contact (2026-09-08)
D46	Training Run-to-Run Variance Measured — M4's Sharpness Gate Is Seed-Fragile (2026-09-08)
D47	Ladder Rungs Are Compared at Parameter Parity; Gradient Accumulation Keeps Batch Out of the Comparison (2026-09-08)
D48	M5 Export and TensorRT Benchmark — §12's Risk Landed on the Baseline, Not the Transformer (2026-09-08)
D49	M5 Negative Result — StructFormer Underperforms the CNN Baseline at Parameter Parity on This Scene (2026-09-08)
D50	M5 Review — The Ladder Letter Was Wrong and the Export Contract Reversed Itself (2026-09-08)
D51	M5 Evidence Audit — A Generator That Could Not Reproduce Its Own Report, and Statistics Stated Beyond What Was Measured (2026-09-08)
D52	Five Review Findings — A Report Path That Overwrote Committed Evidence, and a Mask That Only Discouraged (2026-09-08)
D53	RCS Implemented and Measured — The Headline Metric Does Not Degrade With Time Unseen (2026-09-09)
D54	Strict Masking Reaches TensorRT; RCS Re-measured Over Frame Windows (2026-09-10)
D55	M6 Warp, Mask and Gate — The Validation Split Cannot Measure Reprojection (2026-09-10)
D56	Model D Trains From Scratch — Decision 19's Staged Resume Is a Warm Restart Here (2026-09-10)
D57	M6 Negative Result — Temporal Reprojection Does Not Separate From StructFormer Seed Noise (2026-09-10)
D58	M6 Review — Four Flagged Deviations the Log Never Absorbed, and a Pre-Registration That Contained Its Own Subject (2026-09-10)
D59	M6 Export Evidence — Model D Deploys, at 47 % More Latency and With Cross-Frame State (2026-09-10)
D60	Cross-Frame Teacher Conditioning Is REFUTED — 11x Worse, Not Better (2026-09-12)
D61	Fifteen Sessions — Viewpoint and Trajectory Diversity, Not New Scene Content (2026-09-13)
D62	Early Stopping on Every Rung (2026-09-13)
D63	Model B — the CNN Temporal Rung M6 Was Missing (2026-09-13)
D64	Session-Level Split — the Project's First Independent Test Set (2026-09-13)
D65	Adding Sessions Silently Repointed Every Historical Test (2026-09-14)
D66	Model B's Temporal Fusion Is Inert; Model D's Is Not (2026-09-14)
D67	M6 PASSES on Ladder v2 — the Margin Was Stale, Not the Result (2026-09-16)
D68	The Recorder Does Not Wait on the Compositor (2026-09-20; written 2026-09-28)
D69	Ladder v3 — 29 Sessions, and a Projection Pre-Registered Before the Result (2026-09-20; written 2026-09-28)
D70	Product Scope — a Style Renderer, Not a Game Generator (2026-09-21)
D71	M5 Fails at 32x the Data — the Scaling Hypothesis Is Refuted (2026-09-21)
D72	RCS Cannot Measure EPAM — Its Claim Moves to Sharpness (2026-09-21)
D73	The Residual Memory Probe — Information Exists, but Not at EPAM's Timescales (2026-09-21)
D74	M7 Complete — Structured Entity Awareness (2026-09-21)
D75	Entity-Masked Attention Degenerates to a Lookup; Model E Is a Null Result (2026-09-21)
D76	Entity Consistency Is Keyed by Identity, Not by Slot (2026-09-22)
D77	Entity Conditioning Sharpens — a Predicted Effect, Confirmed at 2.5 Sigma (2026-09-23)
D78	M8 Design — EPAM Store, Model F, and a Built-In No-Memory Control (2026-09-23)
D79	M8 Criterion 4 Not Met — EPAM Memory Does Not Improve Output (2026-09-26)
D80	Model B Was Never Trained With History — Decision 66 Retracted in Cause (2026-09-26)
D81	Entity-Anchored Teacher Noise Does Not Create Persistent Appearance — EPAM v2 Not Pursued (2026-09-26)
D82	The Temporal Trainer Had No Early Stopping; GPU Memory Headroom for the Display (2026-09-26)
D83	M9 Image-Quality Ladder on One Scoring Path (2026-09-27)
D84	Model A's TensorRT Failure Diagnosed and Fixed — GroupNorm's InstanceNormalization (2026-09-27)
D85	Temporal Conditioning Cuts Flicker on Both Backbones — B Works Once Trained Correctly (2026-09-27)

No.	Decision
D86	M10 Live Loop — C++ Runtime, GDExtension, and a Live Capture Path (2026-09-27)
D87	A Training Dataset From the Playable Game (2026-09-27; written 2026-09-28)
D88	Selecting the Game Teacher Style by the Alignment Gates (2026-09-28)
D89	The Naive Raw-ID Control Matches Structured Entity Use on This Scene (2026-09-28)
D90	Game Teacher Targets — Oil Passes, Watercolour Does Not (2026-09-29)
D91	Model B on the Playable Game — Held-Out Results (2026-09-30)
D92	A Bold Watercolour Passes on the Game at Strength 0.55 (2026-09-30)
D93	Bold Watercolour Across All Game Sessions — 17/24 ALIGNED (2026-09-30)
D94	Per-Frame Exclusion for the Two-Style Game Model; Two-Style Model B Built (2026-09-30)
D95	Two-Style Model B on the Game — Held-Out Test Results (2026-10-02)
D96	Live Style Switching in the Playable Game — Working, Without Frame History (2026-10-02)
D97	Live Temporal Path — the Playable Game Now Renders With the Scored Model's History (2026-10-02)
D98	M10 Closed — FP16 Engine, Idle Measurements, Failure Injection, Runtime Log (2026-10-02)
D99	Last Two Open Checks Closed — Game Style Acquisition, and S7 Live (2026-10-23)
D100	Higher-Resolution Game Dataset (768x432, schema m3.0) — Pre-Registered Before Any Data (2026-10-23)
D101	The Teacher's Look Changes With Resolution — Decision 100 Paused Before the Full Run (2026-10-23)
D102	Re-Tuning Strength Cannot Recover Today's Look at 768x432 — Higher Resolution Stopped Before the Bulk Run (2026-10-23)